Experimental plan

I want to test whether smarter attention / routing allows MPNNs solve two-radius better

1 - Compare performance of different models, across varying number of nodes, and dimension.

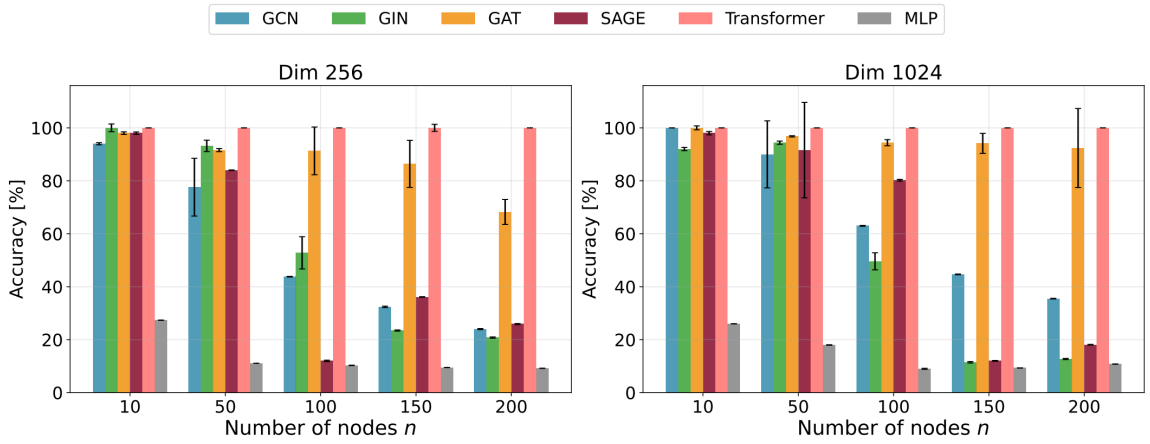
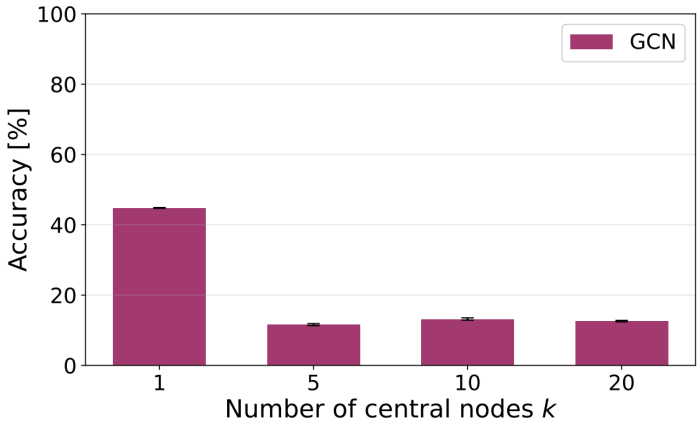


Figure 2: Test accuracy comparison across different models on the Two-Radius problem. Performance is evaluated for varying numbers of nodes $n \in \{10, 50, 150, 200\}$ with hidden dimensions of 256 and 1024. Transformer consistently achieves 100% accuracy while MPNN performance degrades as n increases. Error bars indicate the standard error of the mean.

- Reproduce this exactly this plot, but with TransformerConv

2 - See if increasing number of central nodes can help TransformerConv



(b)

- why is this experiment important?
- while permutation equivariance implies that all central node features are identical across the message-passing process, this is only the case if initial central node features are indistinguishable.
- so give central node features distinct identifiers
- However, conflicting gradients still makes it difficult for the MPNN to utilise the central nodes.
- Attentional flavour GNNs can overcome this by ‘turning off’ edges / ensuring central node only attends to a few target nodes.
- We can do bar chart of convolutional and attentional GCN together in one plot.

I want to test whether smarter structure augmentation / virtual node strategies allows MPNNs solve two-radius better

1 - overall comparison of performance of each of the progressively more complex (or progressively stronger inductive bias) virtual node strategies (using the best hyperparameters for each strategy)

- Since TransformerConv gives the best routing inductive bias, we use this as a base model for all virtual node strategies.
- A baseline is TransformerConv + single Global VN. We do this to compare to the original paper, as the original paper only showed GCN + VN.

▼ We can then do TransformerConv + multiple Global VNs. This is the natural next step since it allows for more routes. We can do a sweep across different numbers of Global VNs and report the performance of the best result.

- for virtual nodes, we can use learnable identifiers (or use fixed one hot encoding identifiers and use MLP to map to hidden dimension).

▼ We then can try TransformerConv + multiple Local VNs. This is the natural next step since this allows for less gradient interference from the target nodes to the virtual ndoes. We will need to sweep across the number of LVNs, and the degree of each. We can report the best performance of this result.

- Local VNs are assigned by random assignment (so that we don't cheat)

- Finally, motivated by the fact that the LVNs may not choose edges between corresponding source and target nodes, we can try probabilistic VNs, where we learn the connections of the VNs. We will need to sweep across number of LVNs, and number of connections. We can report the best performance of this result.
- For a fixed dimension, and for standard two-radius, we can produce a plot like this:

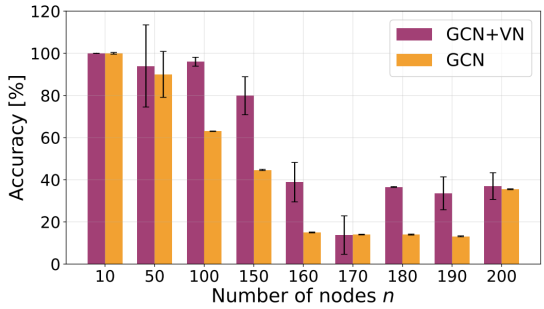
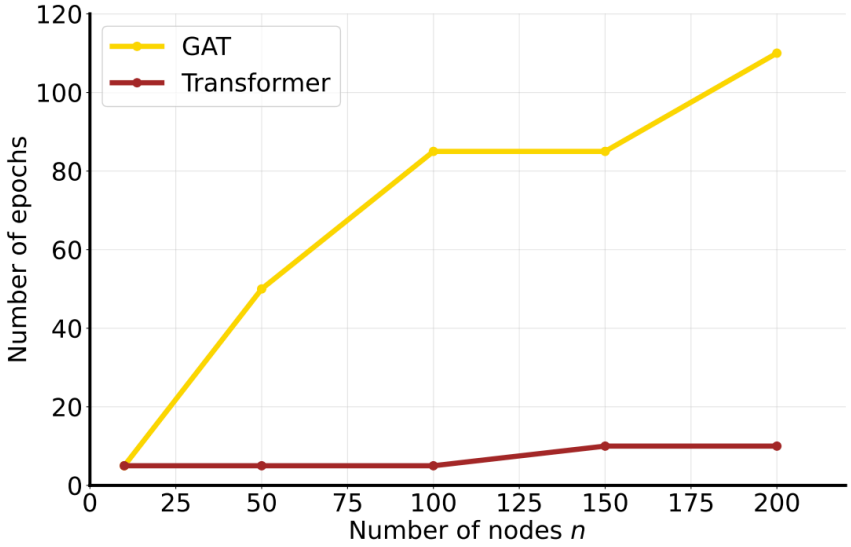


Figure 5: Comparison of GCN performance with and without virtual nodes (VN) on the Two-Radius problem. While virtual nodes provide modest improvements, performance still degrades significantly as n increases, indicating that VNs do not fully address the bottleneck in short-range oversquashing. Error bars indicate the standard error of the mean.

- in our plot we will have bars for: TConv + Global VN, TConv + Multiple Global VN, TConv + Local VN, TConv + Probabilistic VNs.

2 - For the best performing strategy, how easy is it to train compared to transformer?

- it is all well and good if performance is similar, but how difficult is it to train?



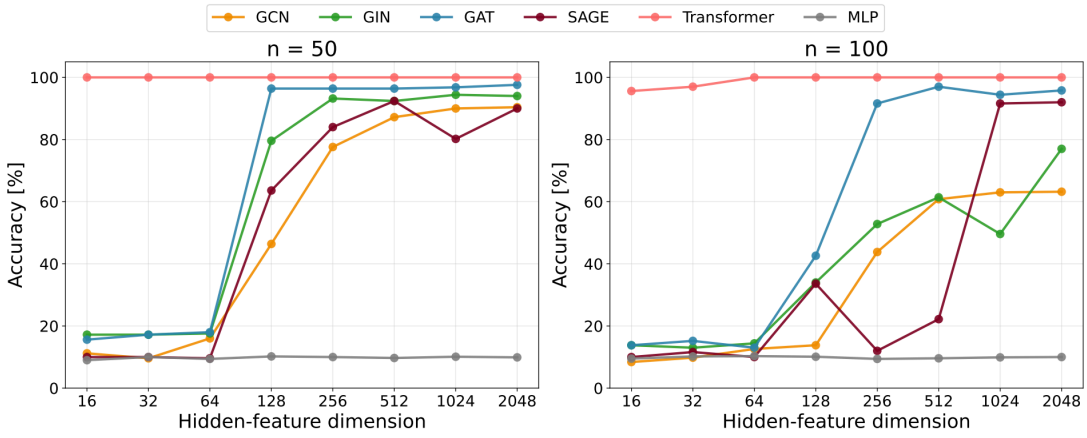
(a)

- Compare TransformerConv + best VN strategy, with Set Transformer
- measured in the number of epochs required to achieve 92% accuracy.
- note that epochs is misleading. Because set transformers compute NxN attention matrix, training is much more expensive per epoch.
- instead of number of epochs, plot accuracy against Wall-Clock Time (seconds) or Total FLOPs
- actually, because two-radius is computationally easy and N is relatively small, The wall-clock time will be entirely dominated by PyTorch overhead rather than actual matrix multiplications.
- so just do epochs. and mention wall clock time in write up.

3 - For the best performing VN strategy, cam more virtual nodes reduce need for higher hidden feature dimension?

- Fine-grained evaluation of the effect of hidden-feature dimension
- keep n fixed, and vary hidden dimension. produce a plot like this

- for best performing VN method, progressively increase number of VNs. We should see that, at each hidden feature dimension, more virtual ndoes does better than less.
- this is because ideally as we increase the number of virtual nodes, we need less hidden dimension, as the virtual nodes will decrease the representional burden of any one node.
- this is exactly what this experiment will test
- this experiment is important because having more virtual nodes is less expensive than increasing hidden dimension for EVERY node.



- actually, it may be best to do this for local virtual nodes (rather than probabilistic nodes). because for probabilistic nodes, as you increase the number of virtual nodes, the MLP which outputs the connections of each node to the virtual node will have a harder time. This optimisation is more difficult, and you might therefore not see increased performance. You want to isolate capacity from ease of optimisation. so if you use local virtual nodes, there is less difficulty in optimisation, and it a more controlled experiment.

Does Attention or VNs really help reduce gradient interference at central nodes? (Digging into why do we see observed results for previous experiments)

- This is the hypothesised mechanism by which I am justifying that attention and virtual nodes help.
- I hypothesise that it reduces the gradient from as many target nodes being backpropagated to the central node(s)
- so we can track gradient of central node in two radius.
- look at the computational graph and extract the gradient of the Loss L with respect to the representation of the central node after the *first* layer
- If the N targets are pulling the central node in N different directions, the vectors will destructively interfere (cancel each other out).
- The magnitude of the surviving gradient vector will be surprisingly small relative to the number of targets.
- IIf you plot the gradient norm vs number of targets, n, a model dufferring from gradient interference at the central node will show the norm staying flat or decaying. However a model with good routing (e.g. TransformerConv + probabilistic VNs) will show a more healthy / stable gradient norm because the irrelevant pathways from target to source are given lower weight by the attention mechanism.
- so basically plot gradient of central node after first layer (as we increase n) and compare GCN with Transformer conv and with Transformer conv + VNs.
- Track the **L2-Norm of h_central.grad** averaged across the batch

▼ a potentially better approach

  - measure gradient of each target node with respect to the central node. do this for all target nodes.
  - then compute:

$$\text{Coherence} = \frac{\left\| \sum_{t=1}^{N} \nabla_{h_c} L_t \right\|}{\sum_{t=1}^{N} \left\| \nabla_{h_c} L_t \right\|}$$

- If all targets pull perfectly in the same direction (no interference), this ratio is 1 (not sure why)
- f targets pull in random, conflicting directions (high interference), this ratio drops towards 0
-

Avoiding hyperparameter hell

- fix layers L=4 (as is done in the original paper)
- fix hidden dimension to e.g. 1024 unless we are testing impact of hidden dimension
- tune hyperparameters on e.g. N=50, rather than on all models. (use e.g. wandb for hyperparam search)